

راهنمای جامع Scikit-Image برای پردازش تصویر با پایتون

مقدمه: آشنایی با Scikit-Image

کتابخانه **Scikit-Image** یک کتابخانه‌ی متن‌باز برای پردازش تصویر در زبان پایتون است که به عنوان بخشی از اکوسیستم SciPy توسعه یافته است ¹ ². این کتابخانه بر پایه‌ی کتابخانه‌های علمی **NumPy** و **SciPy** ساخته شده و از آرایه‌های چندبعدی NumPy به عنوان ساختار داده‌ای تصاویر استفاده می‌کند ¹. SciKit-Image مجموعه‌ی وسیعی از الگوریتم‌های رایج پردازش تصویر را در بر می‌گیرد؛ از جمله الگوریتم‌های مربوط به **قطعه‌بندی تصاویر** (Image Segmentation)، **تبدیلات هندسی** روی تصاویر، **تبدیلات فضای رنگ**، **آنالیز و اندازه‌گیری ویژگی‌های تصاویر**، **فیلترگذاری** و بهبود تصویر، **عملیات مورفولوژی**، **تشخیص ویژگی‌ها** (مانند لبه‌ها، گوشه‌ها، الگوها) و بسیاری موارد دیگر ². این قابلیت‌ها SciKit-Image را به ابزاری قدرتمند برای کاربردهای پژوهشی، آموزشی و حتی صنعتی در حوزه‌ی بینایی کامپیوتر تبدیل کرده است ³ ⁴.

یکی از مزیت‌های مهم SciKit-Image سادگی استفاده از آن حتی برای مبتدیان است. توابع و الگوریتم‌های این کتابخانه با رابط‌های ساده‌ای طراحی شده‌اند و هر کسی با آشنایی مقدماتی با پایتون می‌تواند از آن استفاده کند ⁵. به علاوه، کدهای پیاده‌سازی شده در SciKit-Image از کیفیت بالایی برخوردار بوده و توسط یک جامعه‌ی فعال از برنامه‌نویسان داوطلب به صورت مداوم پشتیبانی و به‌روزرسانی می‌شود ⁵ ⁶. این کتابخانه به صورت رایگان و بدون محدودیت در دسترس است ⁶ و به دلیل داشتن کدهای بازبینی‌شده (Peer-Reviewed) از پایداری و صحت علمی خوبی برخوردار است. کافی است کتابخانه را با نام `skimage` در پایتون **ایمپورت** کنید و آماده‌ی کار شوید ⁷.

از آنجا که SciKit-Image بخشی از **SciKits** (مجموعه ابزارهای علمی مبتنی بر SciPy) است، طراحی آن به گونه‌ای بوده که **سازگاری بالایی با کتابخانه‌های NumPy/SciPy** دارد ². در عمل، ورودی و خروجی تمامی توابع این کتابخانه، آرایه‌های **NumPy** هستند و بنابراین می‌توان به راحتی از قابلیت‌های Numpy برای **دستکاری پیکسل‌ها** و داده‌های تصویر بهره برد ⁸. به عنوان مثال، می‌توان پس از بارگذاری یک تصویر به کمک SciKit-Image، عملیاتی نظیر **برش (Slicing)**، **ماسک‌گذاری (Masking)** یا **تغییر مقادیر پیکسل‌ها** را مستقیماً با قابلیت‌های NumPy انجام داد ⁸. این یکپارچگی با NumPy/SciPy باعث می‌شود SciKit-Image به شکلی طبیعی در کنار سایر ابزارهای علمی پایتون (مانند کتابخانه‌ی **Matplotlib** برای نمایش تصاویر) مورد استفاده قرار گیرد ⁸.

در ادامه‌ی این راهنما، ابتدا روش **نصب و راه‌اندازی SciKit-Image** را مرور کرده و سپس به بررسی مهم‌ترین **ماژول‌ها و قابلیت‌های کلیدی** این کتابخانه خواهیم پرداخت. همچنین با ارائه مثال‌های عملی (از **فیلترگذاری تصاویر** گرفته تا **تشخیص چهره و تطبیق الگو**)، نحوه‌ی به‌کارگیری SciKit-Image در پروژه‌های بینایی کامپیوتر را به صورت گام‌به‌گام نشان می‌دهیم. این راهنما به شکلی تهیه شده که یک فرد علاقه‌مند (در سطح نیمه‌حرفه‌ای تا حرفه‌ای) بتواند پس از مطالعه‌ی آن، استفاده‌ی مؤثر از SciKit-Image در پروژه‌های پردازش تصویر را فرا گیرد.

نصب و راه‌اندازی Scikit-Image

برای شروع کار با Scikit-Image نیاز است که پایتون (ترجیحاً نسخه‌ی ۳.۱۰ یا جدیدتر) را بر روی سیستم خود نصب داشته باشید^۹. سپس می‌توانید با استفاده از یکی از دو روش رایج زیر، کتابخانه‌ی Scikit-Image را نصب کنید:

- **نصب با pip:** ساده‌ترین راه، استفاده از ابزار مدیریت بسته‌ی pip است. قبل از نصب مطمئن شوید pip به‌روز است و سپس دستور زیر را در ترمینال اجرا کنید^{۱۰}:

```
python -m pip install -U scikit-image
```

این دستور آخرین نسخه‌ی Scikit-Image را به همراه تمامی وابستگی‌های مورد نیاز (نظیر NumPy, SciPy و ...) نصب می‌کند. پس از اتمام، می‌توانید با دستور `python -c "import skimage; print(skimage.__version__)"` صحت نصب و نسخه‌ی Scikit-Image نصب‌شده را بررسی کنید^{۱۱}. اگر قصد دارید مجموعه داده‌های نمونه‌ی Scikit-Image (تصاویر پیش‌فرض در `skimage.data`) نیز در حالت آفلاین در دسترس باشند، می‌توانید پس از نصب، دستور `python -c "import skimage; skimage.data.download_all()"` را اجرا کنید که تمام تصاویر نمونه را دانلود می‌کند^{۱۲}.

- **نصب با Anaconda/Conda:** اگر از توزیع Anaconda یا Miniconda استفاده می‌کنید، معمولاً در مخازن آن موجود است^{۱۳}. کافی است یک محیط مجازی (environment) ایجاد کنید و دستور زیر را اجرا نمایید^{۱۴}:

```
conda install -c conda-forge scikit-image
```

این فرمان نسخه‌ی پایدار Scikit-Image را از کانال conda-forge دریافت و نصب می‌کند. استفاده از conda مزیت نصب خودکار وابستگی‌های سنگینی مثل SciPy را دارد و روی سیستم‌عامل‌های ویندوز/مک نیز به خوبی کار می‌کند^{۱۳}. همچنین Scikit-Image از طریق مدیر بسته‌های سیستم (مثل apt در اوبونتو) نیز قابل نصب است، اما ممکن است نسخه‌های موجود در مخازن سیستم قدیمی باشند^{۱۵}. توصیه می‌شود برای داشتن آخرین ویژگی‌ها، از pip یا conda-forge استفاده کنید.

پس از نصب موفق، برای استفاده از کتابخانه کافی است در ابتدای کد خود آن را **ایمپورت** نمایید:

```
import skimage
```

یا اینکه ماژول‌های فرعی مورد نیاز را مستقیماً ایمپورت کنید، برای مثال:

```
from skimage import data, io, filters
```

با این کار می‌توانید بسته به نیاز، از ماژول‌های مختلف Scikit-Image در کد خود بهره ببرید^{۱۶}. در ادامه به معرفی ماژول‌های کلیدی این کتابخانه می‌پردازیم.

ماژول‌ها و قابلیت‌های کلیدی در Scikit-Image

کتابخانه‌ی Scikit-Image به صورت یک بسته‌ی جامع ارائه شده که شامل چندین **زیرماژول (Submodule)** تخصصی است. هر زیرماژول مجموعه‌ای از توابع مربوط به یک جنبه‌ی خاص از پردازش تصویر را فراهم می‌کند.⁷ در این بخش مهم‌ترین زیرماژول‌های Scikit-Image و کاربرد هر کدام را معرفی می‌کنیم:

- **skimage.data**: این ماژول شامل مجموعه‌ای از **تصاویر نمونه و دیتاست‌های کوچک** است که همراه Scikit-Image عرضه می‌شوند.¹⁷ برای مثال `skimage.data.coins()` تصویری از تعدادی سکه را برمی‌گرداند که در بسیاری مثال‌ها برای آزمایش الگوریتم‌ها استفاده می‌شود.¹⁷ همچنین تصاویر معروفی مانند `camera`, `astronaut`, `coffee` و ... در این ماژول قرار دارند. این تصاویر جهت آموزش و تست الگوریتم‌ها بسیار مفید هستند.

- **skimage.io**: شامل توابع **ورودی/خروجی تصویر** است. با استفاده از `skimage.io.imread()` می‌توانید تصاویر را از فایل (یا حتی از URL) بارگذاری کنید و به صورت آرایه‌ی NumPy در اختیار داشته باشید.¹⁸ همچنین با `skimage.io.imshow()` و `skimage.io.imwrite()` می‌توانید تصاویر را نمایش داده و در فایل ذخیره کنید. لازم به ذکر است که Scikit-Image برای خواندن/نوشتن بسیاری از فرمت‌های رایج تصویر، از کتابخانه‌های PIL/Pillow در پشت‌صحنه استفاده می‌کند و می‌تواند اکثر فرمت‌های پشتیبانی‌شده توسط Pillow را بارگذاری نماید.¹⁹

- **skimage.color**: این ماژول توابعی برای **تبدیل فضای رنگ** تصاویر دارد. برای مثال، با `skimage.color.rgb2gray()` می‌توانید تصویر رنگی را به سطح‌خاکستری تبدیل کنید. یا توابعی برای تبدیل بین فضاهای رنگ مختلف (RGB به HSV، RGB به YUV و برعکس) فراهم شده است. این قابلیت زمانی مفید است که مثلاً بخواهید پردازش را در فضای رنگی مناسب‌تری انجام دهید یا تصاویر را تک‌باندی کنید.

- **skimage.filters**: شامل طیف وسیعی از **فیلترهای پردازش تصویر** است.²⁰ این فیلترها می‌توانند برای **تاری کردن (Blur)** تصاویر، **صاف کردن نویز، تشخیص لبه، افزایش کنتراست محلی** و غیره به کار روند. برای نمونه، فیلترهای کلاسیک **گوسی (Gaussian)**، **میانگین، مدیان، لابلاس** و همچنین فیلترهای **تشخیص لبه** مانند **Prewitt, Sobel** و **Canny** در این زیرماژول قرار دارند.²¹ بسیاری از این توابع مستقیماً روی آرایه‌ی تصویر اعمال می‌شوند و آرایه‌ی حاصل فیلترشده (اغلب با نوع داده‌ی اعشاری float) را برمی‌گردانند.

- **skimage.transform**: توابع مربوط به **تبدیلات هندسی** تصویر نظیر **تغییر اندازه (Rescale/Resize)**، **چرخش، برش (Cropping)**، **بیمایش (Warping)** و حتی **تصحیح پرسپکتیو** در این بخش قرار گرفته‌اند. برای مثال `skimage.transform.resize()` جهت تغییر ابعاد تصویر یا `skimage.transform.rotate()` برای چرخاندن تصویر به کار می‌روند. همچنین توابعی برای **پیاده‌سازی تبدیل هاف** (برای تشخیص خطوط و دایره‌ها) و **درون‌یابی تصاویر** در این ماژول وجود دارد.

- **skimage.morphology**: شامل مجموعه‌ای از **عملیات مورفولوژیک** (ریخت‌شناسی ریاضی) برای تصاویر دودویی و سطح‌خاکستری است. این عملیات که معمولاً بر روی تصاویر **باینری** (سیاه و سفید) انجام می‌شوند، برای **حذف نویزهای ذره‌ای، پر کردن حفره‌ها، باز و بسته کردن (Opening/Closing)** نواحی و **استخراج اجزای متصل** کاربرد دارند.²³ ²⁴ توابعی مانند `morphology.binary_dilation` (بسط)، `binary_erosion` (فرسایش)، `binary_opening` و `binary_closing` و همچنین توابعی برای محاسبه **عنصر ساختاری (structuring element)** مثل `morphology.disk()` در این زیرماژول ارائه شده‌اند. از عملیات مورفولوژی می‌توان پس از آستانه‌گذاری تصاویر جهت **پاکسازی نتیجه‌ی قطعه‌بندی** یا **بستن شکستگی‌های کوچک در لبه اشیاء** استفاده کرد.

- `skimage.feature`: این زیرماژول شامل ابزارهای **استخراج ویژگی** و **تشخیص الگو** در تصاویر است ².
برای مثال الگوریتم‌های **تشخیص گوشه** (همچون Harris یا Shi-Tomasi)، توابع **استخراج ویژگی‌های محلی** (نظیر ORB یا SIFT)، **دسکریپتورهای بافت** (مانند LBP) و همچنین **آشکارسازهای شیء** ساده در این بخش قرار دارند. یکی از جذاب‌ترین ابزارهای این ماژول، **تابع تشخیص لبه‌ی Canny** است که لبه‌های باریک و پیوسته‌ای را در تصویر پیدا می‌کند ²². همچنین یک **کلاسیفایر آبشاری (Cascade)** از پیش‌آمخته برای **تشخیص چهره** در این بخش موجود است که امکان شناسایی صورت افراد در تصاویر را به سادگی فراهم می‌کند (در ادامه نمونه‌ای از آن را خواهیم دید).

- `skimage.segmentation`: همانطور که از نامش پیداست برای **قطعه‌بندی تصویر** به کار می‌رود ².
قطعه‌بندی به معنای تقسیم تصویر به نواحی معنادار (مانند جدا کردن اشیاء از پس‌زمینه) است. این زیرماژول شامل الگوریتم‌های متنوعی مانند **آستانه‌گذاری‌های چندسطحی** (Otsu, Li, Yen و ...)، **بخش‌بندی بر اساس ناحیه و مرز** (مانند Watershed) که در ادامه توضیح داده می‌شود، **خوشه‌بندی پیکسل‌ها** (مثل K-means برای تصاویر)، **تفکیک ابرپیکسل‌ها** (SLIC و Quickshift) و ... است. بعلاوه توابعی جهت **برچسب‌گذاری نواحی متصل** (`skimage.measure.label`) و **جداسازی اجزاء هم‌مرز** (`segmentation.clear_border`) برای حذف اشیاء متصل به مرز تصویر ارائه شده که در پردازش پس از قطعه‌بندی سودمند هستند.

- `skimage.measure`: این بخش ابزارهایی برای **اندازه‌گیری و تجزیه و تحلیل کمی تصاویر** دارد ².
مهم‌ترین قابلیت آن، محاسبه‌ی ویژگی‌های هندسی و آماری **نواحی برچسب‌خورده** در تصویر است. برای نمونه، تابع `measure.regionprops` روی خروجی یک تصویر برچسب‌گذاری‌شده (label image) اعمال شده و ویژگی‌هایی نظیر **مساحت** هر ناحیه، **محیط**، **مرکز جرم**، **مستطیل محاط**، **گوسی‌ان مناسبت** و دهه‌ها ویژگی دیگر را برای هر شیء محاسبه می‌کند. این قابلیت زمانی که بخواهید اشیاء شناسایی‌شده را در تصویر تحلیل کنید (مثلاً برای شمارش یا طبقه‌بندی بر اساس اندازه) بسیار کاربردی است.

- **سایر ماژول‌ها:** افزون بر موارد بالا، Scikit-Image دارای زیرماژول‌های دیگری نیز هست. به عنوان مثال `skimage.exposure` برای تنظیم هیستوگرام و **بهبود کنتراست** (مانند روش Equalization یا adaptive equalization) به کار می‌رود ²⁵. ماژول `skimage.restoration` شامل توابع **ترمیم تصویر** نظیر حذف نویز (فیلترهای میانگین رتبه‌ای، بیلترال، NI-means)، حذف تارهای حرکتی، **کاهش نویز با موجک** و ... است. همچنین `skimage.util` یک سری توابع کمکی (utilities) از جمله برای تغییر نوع داده‌ی تصاویر، نرمال‌سازی و تولید نویز دارد. در نهایت، `skimage.draw` برای **رسم اشکال** پایه (خط، دایره، مستطیل و ...) روی آرایه‌های تصویر کاربرد دارد.

به طور خلاصه، Scikit-Image تقریباً هر آنچه برای پردازش کلاسیک تصاویر نیاز دارید را یکجا فراهم کرده است ². در بخش‌های بعدی با چند کاربرد عملی مهم این کتابخانه آشنا می‌شویم و مثال‌هایی از استفاده‌ی هر کدام را ارائه می‌دهیم.

بارگذاری و نمایش تصویر (شروع کار با تصاویر)

نخستین گام در اکثر پروژه‌های پردازش تصویر، **خواندن تصویر ورودی** و آماده‌سازی آن برای پردازش است. در Scikit-Image همان‌طور که اشاره شد، ساده‌ترین روش برای بارگذاری یک تصویر استفاده از تابع `io.imread` است. این تابع مسیر فایل تصویر (یا آدرس URL آن) را دریافت کرده و یک **آرایه‌ی NumPy** سه‌بعدی (برای تصاویر رنگی) یا دوبعدی (برای تصاویر تکرنگ) برمی‌گرداند ¹⁸ ¹⁹. برای نمونه:

```
from skimage import io
image = io.imread('my_photo.jpg')
print(image.shape, image.dtype)
```

در این مثال، `image` یک آرایه‌ی NumPy است که ابعاد آن (ارتفاع، عرض، تعداد کانال رنگ) و نوع داده‌ی آن (مثلاً `uint8`) مشخص می‌کند. انواع داده‌ی رایج برای تصاویر در پایتون `uint8` (عدد صحیح ۸ بیتی بدون علامت بین 0-255) یا `float` (بین 0.0 و 1.0 هستند) ²⁶ ²⁷ . Scikit-Image تصاویر را به هر دوی این فرم‌ها پشتیبانی می‌کند، اما بسیاری از توابع آن روی تصاویر `float` با محدوده‌ی [0,1] عمل می‌کنند. در صورت نیاز می‌توانید با توابعی نظیر `img_as_float` یا `img_as_ubyte` (از `skimage.util`) نوع داده‌ی تصویر را تغییر دهید.

پس از بارگذاری، گام بعدی معمولاً **نمایش تصویر** است تا مطمئن شویم داده به درستی خوانده شده یا برای بررسی‌های دیداری. Scikit-Image یک تابع ساده‌ی `io.imshow()` دارد که تصویر (آرایه) ورودی را در یک پنجره نمایش می‌دهد. با این حال، بسیاری از کاربران ترجیح می‌دهند برای انعطاف بیشتر از کتابخانه‌ی **Matplotlib** استفاده کنند. برای مثال:

```
import matplotlib.pyplot as plt
plt.imshow(image)
plt.title("Input Image")
plt.axis('off')
plt.show()
```

با استفاده از Matplotlib می‌توانید عنوان، محدوده محورها و رنگ‌نگاشت (کلمپ) نمایش را کنترل کنید. توجه داشته باشید که برای تصاویر **سطح خاکستری** (تک کاناله)، Matplotlib به طور پیش‌فرض آنها را به صورت نقشه حرارتی نشان می‌دهد؛ لذا معمولاً از `plt.imshow(image, cmap='gray')` برای نمایش صحیح سیاه‌وسفید استفاده می‌کنیم. ¹⁷ ²⁸ .

همچنین از ماژول `skimage.data` می‌توان برای دریافت تصاویر نمونه استفاده کرد. به طور مثال، کد زیر تصویر نمونه‌ی **دوربین عکاسی** (camera) را لود و نمایش می‌دهد:

```
from skimage import data
img = data.camera() # 512x512 تصویری
plt.imshow(img, cmap='gray')
plt.show()
```

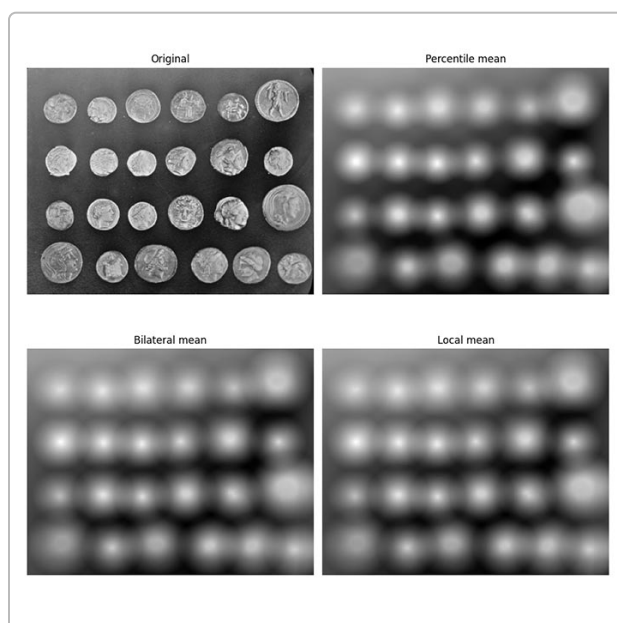
این تصویر تک‌رنگ که به صورت آرایه‌ی `uint8` برمی‌گردد، در بسیاری از آموزش‌ها و الگوریتم‌های پردازش تصویر برای تست به کار می‌رود. به طور مشابه `data.astronaut()` یک تصویر رنگی 512x512 از یک فضاانورد، `data.coins()` تصویری از سکه‌ها روی پس‌زمینه‌ی تیره، `data.cells()` تصویری میکروسکوپی از سلول‌ها و غیره را برمی‌گرداند ¹⁷ ²⁹ . استفاده از این تصاویر آماده به ما اجازه می‌دهد الگوریتم‌های پردازشی را سریع آزمایش کنیم.

نکته‌ی مهم: همان‌طور که تأکید شد **تصاویر در پایتون چیزی جز آرایه‌های NumPy نیستند** ⁸ . بنابراین تمامی عملیات‌های آرایه‌ای NumPy (نظیر برش دادن `image[50:100, 30:80]`، یا محاسبات عددی مانند `image.mean()` و ...) به طور مستقیم روی متغیر `image` قابل انجام است. این ویژگی یک تفاوت عمده‌ی رویکرد

پایتون با مثلاً نرم‌افزار MATLAB در پردازش تصویر است که در آن تصاویر نوع داده‌ی خاص خود را داشتند. در پایتون یادگیری دستکاری آرایه‌ها برابر با یادگیری پردازش مقدماتی تصویر است.

فیلترگذاری و بهبود تصاویر (Image Filtering)

یکی از متداول‌ترین عملیات در پردازش تصویر، **اعمال فیلترهای مختلف** برای بهبود یا استخراج ویژگی‌ها از تصاویر است. منظور از فیلترگذاری اعمال یک عملیات محلی روی تصویر است که باعث تولید تصویر جدیدی می‌شود؛ برای مثال **تار کردن تصویر** برای کاهش نویز، یا **استخراج لبه‌ها** برای برجسته‌سازی مرز اشیاء. Scikit-Image مجموعه‌ی کاملی از این فیلترها (پایین‌گذر، بالاگذر و غیره) را در اختیار ما قرار می‌دهد.²¹



خروجی اجرای چند نوع فیلتر میانگین بر روی تصویر نمونه‌ی سکه‌ها. تصویر اصلی در بالا-چپ، فیلتر "میانگین صدکی" (Percentile Mean) در بالا-راست، فیلتر "میانگین بایلترال" در پایین-چپ و فیلتر "میانگین محلی ساده" در پایین-راست اعمال شده است.³⁰ این فیلترهای مختلف برای حذف نویز در عین حفظ لبه‌های تصویر به کار می‌روند.

در کتابخانه Scikit-Image بسیاری از فیلترهای رایج از طریق توابع موجود در ماژول `skimage.filters` یا `skimage.restoration` قابل دسترسی هستند. به عنوان نمونه‌های پرکاربرد:

- فیلترهای محوکننده (Smoothing):** این فیلترها برای کاهش جزئیات ریز و نویز تصویر به کار می‌روند. فیلتر **گوسی** (`filters.gaussian`) یک ماتریس متداول است که هر پیکسل را با میانگین‌گیری وزنی از همسایگی‌اش (با وزن‌های گوسی) محو می‌کند. فیلتر **میانگین** (`filters.rank.mean`) در زیرماژول `rank` (`filters.uniform`) نیز هر پیکسل را با میانگین همسایه‌ها جایگزین می‌کنند. فیلتر **مدین (Median)** هر پیکسل را با میانه‌ی پنجره‌ی اطرافش عوض می‌کند و برای حذف نویز ضربه‌ای بسیار موثر است. در شکل بالا، ما نمونه‌ای از **فیلترهای میانگین رتبه‌ای** را مشاهده می‌کنیم که با رویکردهای متفاوت (میانگین صدکی، بایلترال، محلی) عمل محو کردن را انجام داده‌اند.

- فیلترهای لبه‌یاب (Edge Detectors):** برای **تشخیص لبه‌ها** (مرزهای اشیاء) در تصویر استفاده می‌شوند. از معروف‌ترین آنها **فیلتر Sobel** (`filters.sobel`) و **فیلتر Prewitt** هستند که تقریبی از مشتق تصویر را در

دو جهت محاسبه کرده و شدت لبه را می‌دهند ³¹ . همچنین **فیلتر لاپلاس** (`filters.laplace`) از دومین مشتق برای مشخص کردن نواحی تغییر شدت استفاده می‌کند. مثال:

```
from skimage import data, filters
image = data.coins() # تصویر نمونه سکه‌ها
edges = filters.sobel(image) # محاسبه لبه‌ها با فیلتر سوبل
plt.imshow(edges, cmap='gray')
plt.show()
```

در این کد، تصویر سکه‌ها به صورت سطح‌خاکستری گرفته شده و فیلتر Sobel بر روی آن اعمال می‌شود تا لبه‌های سکه‌ها استخراج گردد. نتیجه یک تصویر خاکستری جدید (`edges`) است که شدت روشنی در آن نشان‌دهنده احتمال وجود لبه در آن نقطه است ³¹ . (در بخش بعد، روش‌های دقیق‌تر تشخیص لبه مانند **Canny** را نیز خواهیم دید).

- **فیلترهای افزایش جزئیات و وضوح:** گاهی نیاز است تصویر را **شارپ‌تر** کنیم یا جزئیات را برجسته نماییم. در این موارد از فیلترهای **High-Pass** (گذردهنده فرکانس بالا) استفاده می‌شود. یک روش معمول، گرفتن تصویر **گائوسی بلور** شده و تفریق آن از تصویر اصلی است (معروف به **Unsharp Mask** که در `filters.unsharp_mask` پیاده‌سازی شده). این کار لبه‌ها و جزئیات را تقویت می‌کند. همچنین فیلترهایی مانند `filters.rank.enhance_contrast` برای بهبود کنتراست محلی تصویر به کار می‌روند.

- **فیلترهای آستانه‌گذاری تطبیقی:** هرچند آستانه‌گذاری یک روش قطعه‌بندی است، اما می‌توان آن را فیلتر ساده‌ای دانست که تصویر را به دودویی تبدیل می‌کند. توابعی مانند `filters.threshold_otsu` مقدار آستانه بهینه به روش اتسو را برمی‌گردانند ³² ³³ . همچنین توابعی برای آستانه‌گذاری **محلی/تطبیقی** (مثلاً `threshold_local`) وجود دارند که برای هر ناحیه کوچکی از تصویر آستانه جداگانه تعیین می‌کنند تا در تصاویر با نورپردازی ناهمگون، بخش‌های روشن و تاریک هر کدام به‌درستی تفکیک شوند.

به طور کلی، فیلترگذاری یکی از ابزارهای اولیه و اساسی در آماده‌سازی تصاویر است و ترکیب مناسب فیلترها می‌تواند به **بهبود کیفیت تصویر** (مثلاً حذف نویز) یا **استخراج اطلاعات مهم** (مانند لبه‌ها یا گوشه‌ها) کمک شایانی کند ²¹ . در شکل ابتدای این بخش مشاهده کردید که یک تصویر می‌تواند با روش‌های متفاوتی فیلتر شود و هر کدام خروجی متفاوتی ارائه می‌دهند. انتخاب فیلتر مناسب بستگی به هدف شما از پردازش تصویر دارد.

تشخیص لبه‌ها و ویژگی‌ها (Edge & Feature Detection)

تشخیص لبه‌ها یکی از گام‌های کلیدی در بسیاری از سیستم‌های بینایی کامپیوتر است. لبه به مرز بین نواحی مختلف شدت روشنایی در تصویر گفته می‌شود و معمولاً نشان‌دهنده مرز اشیاء است. همان‌گونه که در بخش فیلترها اشاره شد، برخی فیلترها نظیر Sobel و Prewitt قادر به یافتن لبه‌ها هستند. اما روش قدرتمندتری که Scikit-Image در اختیار می‌گذارد، **آشکارساز Canny** است.

الگوریتم **Canny** با چند مرحله پی‌درپی (تاری گوسی، محاسبه گرادیان، نازک‌سازی لبه و هیستریزیس آستانه) لبه‌های پیوسته و نازکی را از تصویر استخراج می‌کند. در Scikit-Image کافیست از تابع `skimage.feature.canny` استفاده کنید ²² . مثلاً:

```
from skimage import feature
edges = feature.canny(image, sigma=1.0)
```

خروجی این تابع یک تصویر دودویی (آرایه‌ی بولین) به اندازه تصویر ورودی است که پیکسل‌های مقدار True نشان‌دهنده‌ی لبه‌های کشف‌شده هستند. پارامتر `sigma` انحراف استاندارد فیلتر گاوسی اولیه را تعیین می‌کند (که روی میزان جزئیات تشخیص داده‌شده اثر می‌گذارد). مزیت Canny نسبت به فیلترهای ساده‌تر این است که لبه‌های متصل تولید می‌کند و تا حدی نسبت به نویز مقاوم است ²².

نقاط گوشه و ویژگی‌های **interest points** نیز در تحلیل تصاویر اهمیت دارند. Scikit-Image توابعی برای **تشخیص گوشه‌ها** ارائه می‌دهد، از جمله `corner_harris` برای محاسبه نقشه Harris corner response و `corner_peaks` برای استخراج مختصات گوشه‌های محتمل از آن نقشه. همچنین الگوریتم FAST (Features from Accelerated Segment Test) با تابع `corner_fast` در دسترس است که گوشه‌ها را با سرعت بالا می‌یابد. پس از یافتن نقاط کلیدی (مثل گوشه‌ها)، می‌توان از **توصیفگرها** برای استخراج بردار ویژگی آن‌ها استفاده کرد (مثلاً `BRIEF` یا `ORB`). در Scikit-Image پیاده‌سازی ORB موجود است که ترکیبی از یک آشکارساز (FAST) و یک توصیفگر چرخش‌ناپذیر و باینری است. استفاده از آن به شکل زیر است:

```
from skimage import feature, color
img_gray = color.rgb2gray(image) # تبدیل به خاکستری
orb = feature.ORB(n_keypoints=100)
orb.detect_and_extract(img_gray)
keypoints = orb.keypoints
descriptors = orb.descriptors
```

در این مثال، ابتدا تصویر به خاکستری تبدیل شده و سپس ORB صد نقطه‌ی کلیدی برتر را شناسایی و توصیف می‌کند. خروجی `orb.keypoints` آرایه‌ای از مختصات گوشه‌های یافت‌شده و `orb.descriptors` ماتریسی شامل توصیفگرهای 256 بیتی (هر کدام به صورت آرایه‌ای از اعداد uint8) برای هر نقطه است. این ویژگی‌ها در کارهایی مثل **تطبیق ویژگی بین دو تصویر** (feature matching) برای یافتن نقاط متناظر بین تصاویر، مثلاً در استیچ‌کردن تصاویر پانوراما کاربرد دارد.

تشخیص الگوها و اشیاء خاص: Scikit-Image ابزارهای ساده‌ای برای شناسایی برخی الگوهای معین نیز دارد. به عنوان مثال، برای **تشخیص دایره‌ها** در تصویر می‌توان از **تبدیل هاف دایره** (`transform.hough_circle`) استفاده کرد. ابتدا لازم است لبه‌های تصویر (مثلاً با Canny) مشخص شوند و سپس تابع `hough_circle` را روی تصویر دودویی لبه اعمال کرد تا دایره‌های احتمالی (با شعاع‌های مشخص) را بیابد. به همین ترتیب برای یافتن خطوط مستقیم (مانند لبه‌های یک جاده در تصویر) کاربرد دارد. `transform.hough_line`

یکی دیگر از قابلیت‌های Scikit-Image، که شاید در نگاه اول غافلگیرکننده باشد، **تشخیص چهره** (Face Detection) است که در زیرمجموعه feature از طریق یک **طبقه‌بند آنبشاری از پیش آموزش‌یافته** فراهم شده است ³⁴ ³⁵. در بخش بعد، این قابلیت را با جزئیات بیشتری بررسی می‌کنیم.

تشخیص چهره با Scikit-Image (مثال کاربردی)

تشخیص چهره در تصاویر، از جمله کاربردهای رایج بینایی کامپیوتر است که معمولاً با استفاده از روش‌هایی مانند **Haar Cascade** یا مدل‌های یادگیری عمیق انجام می‌شود. در Scikit-Image یک مدل آنبشاری یادگیری ماشین کلاسیک بر پایه **الگوی باینری محلی (LBP)** تعبیه شده است که می‌تواند چهره‌های روبه‌رو (frontal face) را شناسایی کند ³⁴. این مدل به صورت فایل از پیش آموزش‌یافته در کتابخانه موجود است و از طریق کلاس `Cascade` قابل استفاده می‌باشد.

برای استفاده از این قابلیت، ابتدا باید فایل مدل را بارگذاری کرده و یک آشکارساز بسازیم، سپس تصویر مورد نظر را به آشکارساز بدهیم تا مکان چهره‌ها را پیدا کند. مثال زیر که از مستندات Scikit-Image الهام گرفته شده، نحوه‌ی انجام این کار را نشان می‌دهد:

```
from skimage import data, feature
import matplotlib.pyplot as plt
from matplotlib import patches

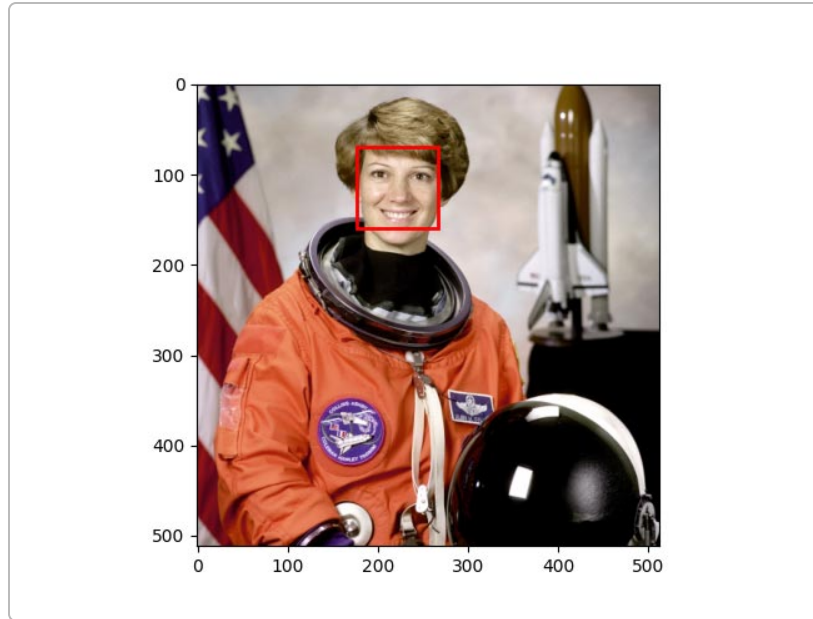
# بارگذاری تصویر نمونه (فضانورد) و مدل تشخیص چهره پیش‌فرض
image = data.astronaut() # 512x512 تصویر رنگی
trained_file = data.lbp_frontal_face_cascade_filename()
detector = feature.Cascade(trained_file)

# اجرای آشکارساز روی تصویر
faces = detector.detect_multi_scale(img=image,
                                   scale_factor=1.2,
                                   step_ratio=1,
                                   min_size=(60, 60),
                                   max_size=(300, 300))

# ترسیم مستطیل دور چهره‌های کشف‌شده
fig, ax = plt.subplots()
ax.imshow(image)
for face in faces:
    r, c, width, height = face['r'], face['c'], face['width'], face['height']
    rect = patches.Rectangle((c, r), width, height,
                             fill=False, color='red', linewidth=2)
    ax.add_patch(rect)

plt.show()
```

در این کد، `data.astronaut()` یک تصویر نمونه شامل چهره‌ی یک فضانورد را برمی‌گرداند. سپس مسیری فایل مدل تشخیص چهره (مبتنی بر LBP) را به ما می‌دهد. با ساختن شیء `feature.Cascade` و دادن این فایل، آشکارساز آماده می‌شود. تابع `detect_multi_scale` روی تصویر اعمال می‌گردد و تمامی نواحی شامل چهره را برمی‌گرداند. در اینجا تعیین می‌کند که رزولوشن تصویر به تدریج ۲۰٪ کوچکتر شود و در هر مقیاس جستجو انجام گیرد، و `max_size / min_size` نیز حدود اندازه‌های قابل قبول برای چهره را مشخص کرده است. خروجی `faces` لیستی از دیکشنری‌هاست که هر کدام مختصات (r,c) و عرض و ارتفاع یک مستطیل چهره‌ی شناسایی‌شده را در بر دارد. در نهایت، با استفاده از Matplotlib یک مستطیل قرمز دور هر چهره رسم کرده‌ایم تا نتیجه‌ی تشخیص را ببینیم. 37 36



نمونه خروجی تشخیص چهره با *Scikit-Image* روی تصویر فضانورد (تصویر نمونه‌ی کتابخانه). کادر قرمز رنگ موقعیت چهره‌ی شناسایی‌شده را نشان می‌دهد. این نتیجه با استفاده از یک مدل آبخاری *LBP* از پیش‌آموزش‌یافته حاصل شده است ³⁶ ³⁷.

همان‌گونه که مشاهده می‌کنید، با چند خط کدنویسی توانستیم یک **تشخیص چهره** ساده اما کاربردی را پیاده‌سازی کنیم. البته باید توجه داشت که این روش یک مدل کلاسیک (Haar-like یا *LBP*) است و دقت آن به اندازه‌ی روش‌های پیشرفته‌ی یادگیری عمیق نیست. با این حال، برای پروژه‌های ساده و آموزشی یا به عنوان نقطه شروع، بسیار مفید است (مزیت اصلی آن نیز عدم نیاز به نصب مدل‌های حجیم و سرعت اجرای بالاست).

تطبیق الگو (Template Matching)

تطبیق الگو تکنیکی برای پیدا کردن محل یک زیرتصویر (الگو) داخل یک تصویر بزرگ‌تر است. برای مثال، فرض کنید تصویری از صحنه‌ای دارید و به دنبال وجود یک شیء خاص (مثلاً لوگو یا قطعه‌ای) در آن تصویر می‌گردید. اگر یک تصویر نمونه از آن شیء داشته باشید، می‌توانید با تطبیق الگو مکان آن را در تصویر اصلی بیابید.

در *Scikit-Image* این قابلیت از طریق تابع `match_template` در ماژول `skimage.feature` ارائه شده است ³⁸. این تابع از **همبستگی متقابل نرمال‌شده** برای پیدا کردن قسمت‌هایی از تصویر که بیشترین شباهت را به الگوی داده‌شده دارند، استفاده می‌کند ³⁸. خروجی `match_template` یک تصویر نقشه‌ی شباهت است که هر پیکسل آن میزان تطابق الگو در آن مکان (در تصویر اصلی) را نشان می‌دهد (مقداری بین 1- تا 1، که 1 به معنای تطابق کامل است).

روش استفاده را با یک مثال شرح می‌دهیم. تصویر نمونه‌ی **سکه‌ها** (`data.coins()`) را در نظر بگیرید و فرض کنید می‌خواهیم جای یک سکه‌ی خاص را در این تصویر پیدا کنیم. ابتدا باید تصویر الگو (`template`) را مشخص کنیم؛ در اینجا می‌توانیم یکی از سکه‌های داخل تصویر را برش بزنیم و به عنوان الگو استفاده کنیم:

```
import numpy as np
from skimage import data, feature
```

```

image = data.coins()           # تصویر اصلی (سکه‌ها)
template = image[170:220, 75:130] # زیرتصویر یک سکه به عنوان الگو
result = feature.match_template(image, template)

```

در این کد، با برش `[170:220, 75:130]` یک سکه از تصویر اصلی جدا شده و به عنوان `template` انتخاب گردیده است (مختصات این برش با دانستن مکان تقریبی یک سکه تعیین شده). سپس `match_template` روی کل تصویر اعمال شده و نتیجه در `result` ذخیره می‌شود. آرایه‌ی `result` کوچک‌تر از `image` است (در هر بعد به اندازه‌ی الگو کوچک‌تر) و هر مقدار آن میزان همبستگی الگو با تصویر در آن offset را نشان می‌دهد.³⁹

گام بعد، پیدا کردن مکان **بیشینه** در تصویر نتیجه است؛ چرا که بیشترین مقدار همبستگی نشان‌دهنده بهترین تطابق می‌باشد. برای این کار می‌توان از تابع `np.unravel_index(np.argmax(result), result.shape)` استفاده کرد تا ایندکس مکانی بیشترین مقدار را بدست آورد:

```

ij = np.unravel_index(np.argmax(result), result.shape)
x, y = ij[::-1] # شده match تبدیل به مختصات (ستون, سطر) گوشه بالا-چپ ناحیه

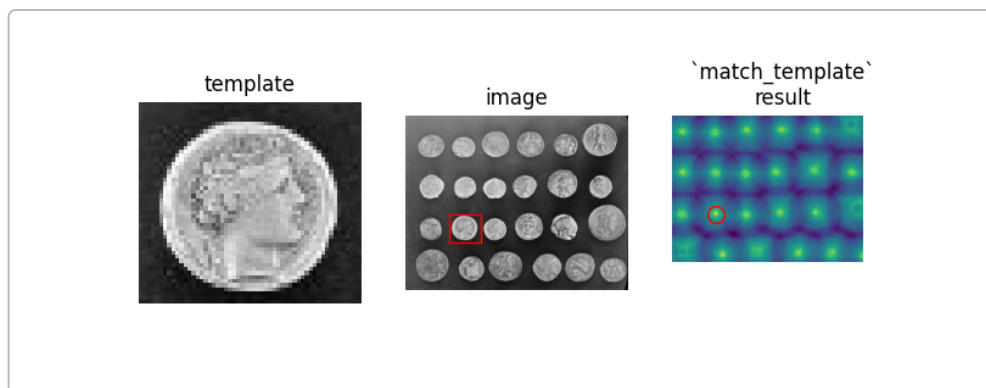
```

متغیر `(x,y)` مختصات نقطه‌ای در تصویر اصلی است که الگو در آن نقطه بیشترین تطابق را داشته (توجه: این مختصات مربوط به گوشه‌ی بالا-چپ الگوی منطبق‌شده است)⁴⁰ ³³. برای نمایش نتیجه می‌توان مستطیلی به ابعاد الگو در آن نقطه رسم کرد:

```

import matplotlib.pyplot as plt
fig, (ax1, ax2, ax3) = plt.subplots(ncols=3, figsize=(8,3))
ax1.imshow(template, cmap='gray')
ax1.set_title('Template')
ax1.axis('off')
ax2.imshow(image, cmap='gray')
ax2.set_title('Image')
ax2.axis('off')
# رسم کادر قرمز در محل تطابق
h, w = template.shape
rect = plt.Rectangle((x, y), w, h, edgecolor='r', facecolor='none')
ax2.add_patch(rect)
ax3.imshow(result, cmap='viridis')
ax3.set_title('Match result')
ax3.axis('off')
# علامت‌گذاری نقطه‌ی ماکزیمم روی نقشه‌ی نتیجه
ax3.autoscale(False)
ax3.plot(x, y, 'ro')
plt.show()

```



نمایش فرآیند تطبیق الگو بر روی تصویر سکه‌ها. تصویر سمت چپ الگو (یک سکه‌ی تک) را نشان می‌دهد. تصویر وسط تصویر اصلی را نشان می‌دهد که محل تطابق با یک مربع قرمز علامت‌گذاری شده است. تصویر سمت راست خروجی تابع `match_template` (نقشه‌ی نمره‌ی تطابق) را نمایش می‌دهد که نقطه‌ی دارای بیشترین مقدار (مشخص‌شده با دایره قرمز) متناظر با مکان الگو در تصویر اصلی است ⁴¹ ⁴².

همان‌طور که مشاهده می‌کنید، الگوریتم تطبیق الگو به درستی مکان سکه‌ی هدف را در تصویر پیدا کرده است. البته باید توجه داشت که **تطبیق الگو نسبت به تغییر مقیاس یا چرخش حساس است**؛ یعنی اگر الگو از لحاظ اندازه یا زاویه با شیء داخل تصویر فرق داشته باشد، `match_template` قادر به شناسایی آن نخواهد بود ⁴¹. برای پوشش مقیاس‌های مختلف، می‌توان تصویر را در چند رزولوشن (مثلاً با `transform.rescale`) جستجو کرد. اما برای حالت کلی (مقیاس و چرخش متغیر)، روش‌های پیشرفته‌تری مانند استفاده از توصیفگرهای ویژگی (ORB، SIFT و ...) توصیه می‌شود.

قطعه‌بندی تصاویر (Image Segmentation)

قطعه‌بندی به فرایند **جداسازی اشیاء مورد نظر از پس‌زمینه** یا تفکیک قسمت‌های مختلف یک تصویر بر اساس ویژگی‌های پیکسل‌ها گفته می‌شود ²⁶. نتیجه‌ی قطعه‌بندی معمولاً یک **نقشه‌ی برچسب** (Label Map) است که در آن به هر پیکسل یک برچسب (مثلاً "شیء" یا "پس‌زمینه") و یا شناسه‌ی شیء مربوطه (اختصاص یافته است. قطعه‌بندی مرحله‌ی بسیار مهمی در بینایی کامپیوتر محسوب می‌شود، زیرا بسیاری از تحلیل‌های بعدی (شمارش اشیاء، اندازه‌گیری ویژگی‌ها، تشخیص اشیاء و ...) بر آن مبتنی هستند.

Scikit-Image ابزارها و الگوریتم‌های گوناگونی را برای انجام انواع قطعه‌بندی فراهم کرده است. ساده‌ترین شکل قطعه‌بندی، **آستانه‌گذاری** (Thresholding) است که تصویر سطح خاکستری را به یک تصویر دودویی (دو بخش: پایین آستانه = پس‌زمینه، بالا آستانه = شیء) تبدیل می‌کند. برای تعیین آستانه‌ی مناسب روش‌های متعددی وجود دارد که معروف‌ترین آنها روش **اُتسو (Otsu)** است. تابع `filters.threshold_otsu(image)` مقدار آستانه بهینه را بر اساس پیشینه‌سازی واریانس بین‌کلاسی محاسبه می‌کند ³². سپس می‌توان به سادگی با مقایسه‌ی تصویر با این آستانه، ماسک دودویی اشیاء را به دست آورد:

```
from skimage.filters import threshold_otsu
thresh_value = threshold_otsu(image_gray)
binary_mask = image_gray > thresh_value
```

این روش در صورتی خوب عمل می‌کند که توزیع شدت روشنایی اشیاء و پس‌زمینه متفاوت باشد (هیستوگرام دو-قلو). اما چالش‌هایی نظیر **نورپردازی غیر یکنواخت** یا **هم‌پوشانی هیستوگرام اشیاء و پس‌زمینه** می‌توانند کارایی

آستانه‌گذاری ساده را محدود کنند⁴³ . به عنوان مثال، تصویر `data.coins()` را در نظر بگیرید که چند سکه‌ی خاکستری‌رنگ روی یک پس‌زمینه‌ی تقریباً تیره قرار دارند. در این تصویر، شدت برخی از قسمت‌های پس‌زمینه با بخش‌هایی از سکه‌ها همپوشانی دارد و همچنین تابش نور باعث شده روشنی سکه‌ها یکنواخت نباشد. بنابراین آستانه‌گذاری ساده منجر به ترکیب شدن بعضی سکه‌ها با پس‌زمینه یا از بین رفتن بخش‌هایی از سکه‌ها می‌شود⁴³ .⁴⁴

برای غلبه بر این مشکل، Scikit-Image پیشنهاد می‌کند که از روش‌های پیشرفته‌تری مثل **استفاده از گرادیان (لبه‌ها) و عملیات مورفولوژی** بهره ببریم⁴⁵ . در مثال سکه‌ها، یک رویکرد این است که ابتدا با استفاده از آشکارساز **Canny** لبه‌های سکه‌ها را بیابیم و سپس با عمل **پر کردن حفره‌ها** ، داخل ناحیه‌ی هر سکه را پر کنیم²² . کد زیر این ایده را پیاده می‌کند:²³

```
from skimage import feature, morphology
import scipy.ndimage as ndi

edges = feature.canny(image_gray, sigma=2.0) # لبه‌یابی با کنی
filled_coins = ndi.binary_fill_holes(edges)
# پر کردن داخل نواحی محصور شده توسط لبه‌ها
cleaned_coins = morphology.remove_small_objects(filled_coins, min_size=20)
```

ابتدا لبه‌های تصویر با Canny (و سیگمای نسبتاً بزرگ برای کاهش نویز) استخراج می‌شوند. سپس تابع کمکی `binary_fill_holes` از SciPy (قابل دسترس با `ndi` که مخفف ndimage است) برای پر کردن فضای داخلی حلقه‌های لبه استفاده می‌شود²³ . نتیجه در تصویر `filled_coins` یک ماسک دودویی است که بیشتر سکه‌ها را کامل پوشش داده، هرچند ممکن است همچنان نویزها یا بخش‌های کوچکی از پس‌زمینه نیز پر شده باشند²⁴ . به همین دلیل از `morphology.remove_small_objects` استفاده شده تا آن اجسام کوچک ناخواسته (با مساحت کمتر از ۲۰ پیکسل) حذف گردند. نهایتاً `cleaned_coins` یک نقشه‌ی دودویی تمیز از سکه‌ها است.

با این حال، حتی این روش نیز ممکن است در شرایطی شکست بخورد؛ مثلاً اگر لبه‌ی یک شیء به طور کامل **بسته** نشود (یک منفذ کوچک داشته باشد)، عمل پر کردن حفره آن ناحیه را پر نخواهد کرد⁴⁶ . در تصویر سکه‌ها نیز شاید یکی دو سکه به طور کامل قطعه‌بندی نشوند. برای غلبه بر این مورد، می‌توانستیم قبل از پر کردن حفره، لبه‌ها را کمی **گسترش (dilate)** دهیم تا شکاف‌ها بسته شوند. اما به جای این کار، سراغ یک روش قدرتمندتر می‌رویم: **قطعه‌بندی بر مبنای ناحیه (Region-based)**.

یکی از روش‌های ناحیه‌مبنا **Watershed** است. ایده‌ی اصلی الگوریتم Watershed این است که تصویر شدت را به عنوان یک توپوگرافی (نقشه‌ی ارتفاع) در نظر بگیریم؛ با انتخاب چند نقطه‌ی **marker** (نشانه) اولیه برای نواحی مورد نظر (مثلاً قطعاً شیء، قطعاً پس‌زمینه)، آنگاه از این نشانه‌ها شروع به "آب‌گیری" می‌کنیم تا حوزه‌های نفوذ هر نشانه مشخص شود⁴⁷ . خطوط برخورد این حوزه‌ها تبدیل به مرزهای قطعه‌بندی خواهند شد⁴⁷ . در عمل، برای اعمال watershed به دو چیز نیاز داریم: یکی تصویر گرادیان یا هر نقشه‌ای که مرزها را برجسته کند (نقشه ارتفاع)، و دیگری تصویر نشانه‌ها (markers) که پیکسل‌های مطمئن از هر کلاس (مثلاً برچسب ۱ برای پس‌زمینه، برچسب ۲ برای شیء) را معین کند.

در مثال سکه‌ها، یک انتخاب خوب برای نقشه‌ی ارتفاع می‌تواند **قدر مطلق گرادیان** تصویر باشد، چون در مرز سکه‌ها مقدار بالایی دارد. از فیلتر Sobel می‌توان برای تقریب گرادیان استفاده کرد⁴⁸ :

```
from skimage import filters, segmentation
elevation_map = filters.sobel(image_gray) # نقشه ارتفاع: شدت گرادیان تصویر
```

حال برای ساخت markers: می‌توانیم بر اساس شدت روشنایی تصویر اصلی، آنهایی که قطعاً تیره هستند را به عنوان پس‌زمینه و آنهایی که قطعاً روشن‌اند به عنوان شیء علامت‌گذاری کنیم ⁴⁹ . مثلاً در تصویر سکه‌ها، پس‌زمینه بسیار تاریک است و سکه‌ها نسبتاً روشن. بنابراین:

```
markers = np.zeros_like(image_gray)
markers[image_gray < 30] = 1 # پیکسل‌های خیلی تیره -> برچسب 1 (پس‌زمینه)
markers[image_gray > 150] = 2 # پیکسل‌های خیلی روشن -> برچسب 2 (شیء/سکه)
```

اکنون markers یک آرایه‌ی عدد صحیح با مقادیر 0، 1 یا 2 است که در آن 1 معرف پس‌زمینه‌ی مطمئن و 2 معرف شیء مطمئن است (پیکسل‌های با مقدار 0 نامشخص‌اند) ⁴⁹ ⁵⁰ . اجرای الگوریتم watershed با استفاده از این اطلاعات ساده است:

```
segmentation_result = segmentation.watershed(elevation_map, markers)
```

این تابع خروجی را به صورت یک آرایه‌ی برچسب‌دهی برمی‌گرداند که هر ناحیه‌ی مجزا یک شماره برچسب منحصر به فرد دارد ⁵¹ . در مسئله‌ی ما احتمالاً پس‌زمینه برچسب 1، همه‌ی سکه‌ها برچسب 2 (چون از ابتدا دو marker داشتیم) و یا در صورتی که سکه‌ها جدا شوند، ممکن است برچسب‌های 2,3,4,... بگیرند. به هر حال، انتظار داریم نتیجه به درستی نواحی تمام سکه‌ها را جدا کرده باشد ⁵¹ .

مزیت روش watershed این است که حتی اگر لبه‌ی شیئی کامل نباشد، با حضور marker داخل آن شیء، آب از آنجا شروع شده و مرز دقیق‌تری نسبت به روش fill holes پیدا می‌شود. در مثال سکه‌ها مشاهده می‌شود که watershed همه‌ی سکه‌ها را تفکیک می‌کند و دیگر پس‌زمینه‌ی مزاحمی باقی نمی‌ماند ⁵¹ . البته این روش نیز چالش‌های خود را دارد: مثلاً **markers نامناسب** می‌توانند منجر به ادغام یا خرد شدن بیش از حد نواحی شوند. انتخاب markers به صورت خودکار یک مبحث پژوهشی است (روش‌هایی مانند threshold گیری چندگانه، distance transform برای objetos متصل، و ... وجود دارند). در Scikit-Image توابع کمکی نظیر `segmentation.clear_border` (حذف نواحی چسبیده به مرز تصویر که معمولاً متعلق به پس‌زمینه‌اند) یا `morphology.label` (برای برچسب‌گذاری نواحی متصل در یک تصویر دودویی به عنوان markers) وجود دارند که می‌توانند پیش‌پردازش markers را آسان‌تر کنند.

خلاصه اینکه، Scikit-Image راهکارهای متنوعی برای قطعه‌بندی در اختیار شما قرار می‌دهد: از روش‌های ساده‌ی آستانه‌گذاری گرفته ⁴³ تا الگوریتم‌های پیشرفته‌تری چون watershed، **خوشه‌بندی** و حتی روش‌های مبتنی بر گراف مانند **Random Walker** یا **Normalized Cuts** . شما می‌توانید بسته به ماهیت مسئله، روش مناسب را انتخاب کرده و سپس در صورت نیاز با عملگرهای مورفولوژی و توابع کمکی، خروجی را تمیزتر و کاربردی‌تر نمایید.

جمع‌بندی

در این راهنمای جامع، ما با کتابخانه‌ی قدرتمند **Scikit-Image** برای پردازش تصویر در پایتون آشنا شدیم و مراحل مختلف کار با آن را بررسی کردیم؛ از **نصب و راه‌اندازی** گرفته تا معرفی **ماژول‌های کلیدی** و پیاده‌سازی چند **پروژه‌ی نمونه** در حوزه‌ی بینایی کامپیوتر. دیدیم که Scikit-Image به علت پوشش گسترده‌ی الگوریتم‌های پردازشی (فیلترهای گوناگون، تشخیص لبه و ویژگی، قطعه‌بندی، آنالیز اشیاء و ...) و نیز **رابط کاربری ساده و یکپارچه با کتابخانه‌های**

NumPy/SciPy ، یک انتخاب عالی برای کارهای تحقیقاتی و آموزشی در زمینه پردازش تصویر است ³ ⁵ . این کتابخانه از سوی یک جامعه فعال پشتیبانی می‌شود و کدهای آن از لحاظ صحت علمی و کارایی به خوبی بررسی شده‌اند ⁵ ⁶ .

برای یادگیری بیشتر و دیدن نمونه‌های عملی متنوع، منابع آنلاین فراوانی در دسترس است. مستندات رسمی Scikit-Image شامل یک **گالری مثال‌ها** است که کد کامل بسیاری از کاربردهای این کتابخانه (از ساده تا پیچیده) را در خود دارد ⁵² ⁵³ . همچنین درسنامه‌های معتبری مانند **یادداشت‌های SciPy** فصل مجزایی را به آموزش Scikit-Image اختصاص داده‌اند ⁵⁴ . پیشنهاد می‌شود برای تسلط بیشتر، این منابع را مطالعه کرده و دست به کد شوید.

امید است این راهنما نقشه‌ی راه روشنی برای شروع کار با Scikit-Image و انجام پروژه‌های پردازش تصویر در پایتون به شما ارائه کرده باشد. اکنون شما می‌توانید با استفاده از این ابزار، انواع پردازش‌های رایج تصویر (از بهبود کیفیت گرفته تا شناسایی الگوها و اشیاء) را انجام دهید و نتایج مورد نظر خود را کسب کنید. موفق باشید!

منابع و لینک‌های مفید:

- مستندات رسمی Scikit-Image و گالری مثال‌ها ⁵² ⁵³ .
- درسنامه پردازش تصویر در پایتون (مجله فرادرس) ³ ²¹ .
- معرفی و آموزش Scikit-Image در وبلاگ Python Market ¹ ⁵⁵ .
- SciPy Lecture Notes – فصل پردازش تصویر با scikit-image ¹³ ⁵⁴ .
- مقاله علمی معرفی Scikit-Image (van der Walt et al., PeerJ 2014) ⁵⁶ .

¹ ¹⁶ ¹⁷ ²⁸ ²⁹ ³⁰ ³¹ ³⁴ ³⁶ ³⁷ ⁵² ⁵⁵ کتابخانه Scikit-image : آموزش پردازش تصویر در پایتون با Scikit-

image – پایتون مارکت، فروشگاه بزرگ پایتون در ایران

[/https://pythonmarket.ir/scikit-image](https://pythonmarket.ir/scikit-image)

² scikit-image - Wikipedia

<https://en.wikipedia.org/wiki/Scikit-image>

³ ⁴ ⁵ ⁷ ⁸ ²¹ پردازش تصویر با پایتون — راهنمای کاربردی - فرادرس - مجله

[/https://blog.faradars.org/image-processing-in-python](https://blog.faradars.org/image-processing-in-python)

⁶ ⁵⁶ scikit-image: Image processing in Python — scikit-image

[/https://scikit-image.org](https://scikit-image.org)

⁹ ¹⁰ ¹¹ ¹² ¹⁴ ¹⁵ Installing scikit-image — skimage 0.25.2 documentation .1

https://scikit-image.org/docs/0.25.x/user_guide/install.html

¹³ ¹⁸ ¹⁹ ²⁰ ²⁷ ⁵⁴ Scikit-image: image processing — Scipy lecture notes .3.3

[/http://scipy-lectures.org/packages/scikit-image](http://scipy-lectures.org/packages/scikit-image)

²² ²³ ²⁴ ²⁵ ²⁶ ⁴³ ⁴⁴ ⁴⁵ ⁴⁶ ⁴⁷ ⁴⁸ ⁴⁹ ⁵⁰ ⁵¹ Image Segmentation — skimage 0.25.2 .11.1

documentation

https://scikit-image.org/docs/0.25.x/user_guide/tutorial_segmentation.html

³² ³³ ³⁵ ³⁸ ³⁹ ⁴⁰ ⁴¹ ⁴² ⁵³ Template Matching — skimage 0.25.2 documentation

https://scikit-image.org/docs/0.25.x/auto_examples/features_detection/plot_template.html